

Real-world (config + logging together) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Basic Version

Approach 1 — the direct way

```
# config_logging_wiring_practice.py
from config import load_config
from logging_setup import get_logger

config = load_config()
logger = get_logger("main")

logger.info("Loaded config with log level: " + config.log_level)
```

Expected output (terminal, assuming `config.log_level` is `"DEBUG"`):

```
Loaded config with log level: DEBUG
```

The exact text depends on what's actually in your config file — this shows the shape of the output, not a fixed value. If `load_config()` fails (a required setting is missing), this version crashes with a raw Python traceback — fine for a first working version, but not what you'd want a user to see.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — f-string, no failure handling yet

```
# config_logging_wiring_practice.py
from config import load_config
from logging_setup import get_logger

config = load_config()
logger = get_logger(__name__)

logger.info(f"Loaded config with log level: {config.log_level}")
```

Expected output (terminal, assuming `config.log_level` is `"DEBUG"`):

```
Loaded config with log level: DEBUG
```

Approach 2 — with the startup failure handled

If config loading can fail, catch that and stop the program with a clear message instead of a scary error dump.

```
# config_logging_wiring_practice.py
import sys

from config import load_config
from exceptions import MissingConfigError
from logging_setup import get_logger

try:
    config = load_config()
except MissingConfigError as e:
    print(f"Startup failed: {e}")
    sys.exit(1)

logger = get_logger(__name__)

logger.info(f"Loaded config with log level: {config.log_level}")
```

Expected output when config is valid: identical to Approach 1.

Expected output when a required config value is missing (assuming `MissingConfigError` 's message names the missing key `API_KEY`):

```
Startup failed: missing required setting: API_KEY
```

and the process exits with status code `1` — no traceback, no logger involved (there isn't one yet at this point).

Difference from Basic: both Intermediate approaches switch to an f-string and `get_logger(__name__)` (so the logger is named after whichever module actually created it, not a hardcoded `"main"`). Approach 2 additionally wraps the one call that can fail — `load_config()` — in a `try / except`, so a missing setting produces one clear line and a clean exit instead of a multi-line traceback a non-technical user would have to decode.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — the minimal version, as a baseline

Shown again here so all 3 Advanced approaches sit side by side — this is the same wiring as Intermediate Approach 1, with no failure handling yet, included as the baseline the next two approaches build on.

```
# config_logging_wiring_practice.py
from config import load_config
from logging_setup import get_logger

config = load_config()
logger = get_logger(__name__)

logger.info(f"Loaded config with log level: {config.log_level}")
```

Expected output (terminal, assuming `config.log_level` is `"DEBUG"`):

```
Loaded config with log level: DEBUG
```

Approach 2 — clean startup-failure handling, one line

```
# config_logging_wiring_practice.py
import sys

from config import load_config
from exceptions import MissingConfigError
from logging_setup import get_logger

try:
    config = load_config()
except MissingConfigError as e:
    sys.exit(f"Startup failed: {e}")

logger = get_logger(__name__)

logger.info(f"Loaded config with log level: {config.log_level}")
```

Expected output when a required config value is missing:

```
Startup failed: missing required setting: API_KEY
```

printed to stderr, process exits with status `1` — same user-visible result as Intermediate Approach 2's `print()` + `sys.exit(1)` pair, but written as one line: `sys.exit(some_string)` prints that string to stderr and exits with code `1` automatically.

Approach 3 — wrapped in `main()`, with the startup event itself logged

```
# config_logging_wiring_practice.py
import sys

from config import load_config
from exceptions import MissingConfigError
from logging_setup import get_logger

def main() -> None:
    try:
        config = load_config()
    except MissingConfigError as e:
        sys.exit(f"Startup failed: {e}")

    logger = get_logger(__name__)
    logger.info(
        "startup complete",
        extra={"log_level": config.log_level, "event": "startup"},
    )

if __name__ == "__main__":
    main()
```

Expected output (terminal), assuming `get_logger()` attaches a plain handler with no custom formatter — same as the Basic version from the `logger_levels` exercise:

```
startup complete
```

Note what this does *not* show: the `extra={"log_level": ..., "event": "startup"}` values don't appear in that line. `extra=` attaches those fields to the log record itself, not to the printed text — a plain formatter only prints `%(message)s`, so it ignores them. They're there for a formatter (or a log-processing tool) that's written to read them, typically a JSON log formatter that would output

something like `{"message": "startup complete", "log_level": "DEBUG", "event": "startup", ...}` instead of a plain sentence. This is what "structured logging" means in practice: the data exists as real fields a machine can filter on, not just as words inside a sentence a human has to parse.

Difference from Intermediate, and between these 3 Advanced approaches: Intermediate Approach 2 already handles the failure path correctly with `print() + sys.exit(1)` — that's genuinely fine to ship. Approach 1 here is the same happy-path wiring shown again as a baseline. Approach 2 tightens the failure path to Python's `sys.exit(message)` idiom — same behavior, one line instead of two. Approach 3 is the real jump: wrapping startup in `main()` behind `if __name__ == "__main__":` means this file becomes safely importable elsewhere (its top-level code doesn't run on import), and logging a "startup complete" event — with structured `extra=` data — means a successful start leaves a trace too, not just failures. That trace is what you'd search for in production logs to answer "was this instance actually running with the config we expected?"

Which one should you actually write? For the exercise itself, Intermediate Approach 2 (try/except around `load_config()`, clean exit, no `main()` yet) is completely correct and what most people should default to — it satisfies "handle the startup failure" without extra machinery. Advanced Approach 2's one-line `sys.exit(message)` is a small, genuinely nicer idiom worth using once you know it. Advanced Approach 3's `main()` wrapper and structured startup logging are worth the extra lines specifically once this script might ever be imported by something else, or once you're running in an environment where logs are collected centrally (a log aggregator, `systemd`, a container platform) and "did this actually start with the config I think it did" is a question you'll really need to answer later.